# The protocol between a trading engine and your backtester

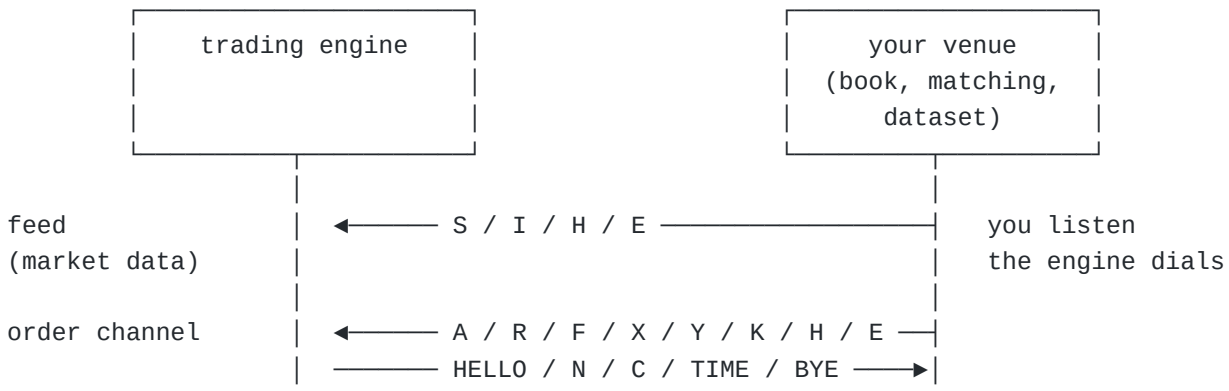
You are writing a **venue**: an exchange simulator with a book and a matcher of your own. A **trading engine** — the same one that runs against a live market — connects to it and trades for real. Your job is to hand it market data out of a dataset and to execute its orders honestly.

This document is the complete specification of how the two of you talk. Next to it are `cmf_driver.hpp`, an implementation of the transport and the clock discipline you can take as it is, and `example_venue.cc`, a small but complete working venue that is the easiest place to start.

## Contents


1. The whole picture
2. How you are launched
3. Channels, framing, encoding
4. Every message
5. Rules per message
6. Time and synchronisation
7. Protocol errors
8. The driver
9. Integration checklist


## 1. The whole picture




**You are the server.** You listen on two TCP ports and the engine connects to both. Not the other way round: the engine starts you itself and dials in immediately, so bind **before** you load your dataset.

**Two connections, not one.** The feed and the order channel are independent. There is no ordering between two TCP connections — a message sent first on one may arrive after a message sent second on the other. The

whole of section 6 grows out of that.

What makes this different from "replay a dataset and add it up": the engine makes its decisions from the book and you execute them, and both sides have to agree on **when** that happened. Without that agreement a run is not reproducible — the result depends on how busy the machine was.

## 2. How you are launched

The engine starts you as a child process and owns your lifetime.

### 2.1 Where you live

Your built binary must be called `backtest` and must sit **next to the engine's binary**. The engine resolves it from its own path, not from `PATH` and not from the current directory, so the engine itself can be started from anywhere.

### 2.2 Arguments

```
backtest --host=<ip> --feed-port=<n> --order-port=<n> [anything else...]
```

| argument | meaning |
|---|---|
| `--host=<ip>` | the address **you listen on**, for both ports |
| `--feed-port=<n>` | the feed port |
| `--order-port=<n>` | the order channel port |
| everything else | passed through; the engine does not look at it |

The first three always arrive, and they come from the engine itself: it names the exact address it is about to dial. There is no second place where this could be set differently — if there were, a disagreement would look exactly like a venue that failed to start.

Everything else reaches you unchanged and in the same order. Usually these are dataset paths. What a dataset is, is yours to decide; the engine knows nothing about it and should not.

**An unknown argument is better ignored than fatal.** The engine may grow a new flag before your build has heard of it. `cmf_driver::ParseOptions` does exactly that.

### 2.3 Lifecycle

- **You are killed together with the engine.** The process is marked so that the parent dying — even abnormally — takes you with it.
- **Ctrl-C in the terminal does not reach you**: you are in a process group of your own. Only the engine stops you.
- **You get `SIGTERM` and five seconds** to stop, then `SIGKILL`. If writing your own results takes longer than that, write them as you go rather than at the end.

- **Your exit does not end the run.** The window lets you run ahead, so by the time you `return` the engine may still be on the hundredth event out of a thousand — it will work through everything you sent. It declares the run over itself, once it has `E` on both channels.
- **Which is why `E` on both channels is mandatory** (section 5.4). Without it the engine cannot tell a finished dataset from a venue that died: it waits, nothing arrives, and the run ends on a silence timeout — recorded as a run whose channels were never closed.

### 2.4 Startup order

The engine starts you and then connects. There is some head start between the two, but its size is not specified and you must not rely on it:

1. bind both ports;
2. **only then** load the dataset, build indexes and everything else;
3. accept the connections (the engine opens the order channel first).

`cmf_driver::Channel` binds in its constructor for exactly this reason.

## 3. Channels, framing, encoding

- **Transport** is TCP. It loses nothing and reorders nothing; anything that looks like a loss is somebody's bug.
- **A frame** is a line terminated by `\n`. A trailing `\r` is accepted and discarded.
- **The field separator** is `|`. It cannot appear inside a field.
- **The encoding** is ASCII. No binary fields, no byte order.
- **Integers** are decimal, unsigned, up to `2^64-1`. Parse with `strtoull` or `from_chars`; `strtoll` saturates silently and parsing via `double` rounds silently. Real data contains values like `10998939501827269164`.
- **Prices** are decimal, never scientific notation. Details, and why it matters, in 5.5.
- **One line is one message.** Nothing is ever spread across several lines.

Read line by line with accumulation: TCP hands you bytes, not lines, and one read can bring you a line and a half, or three and a half. This is the number one source of bugs.

## 4. Every message

### 4.1 Feed: venue → engine

```
S|<seq>|<ts>|<class>|<sec>|<nbid>|<nask>|<px>:<qty>|…     book snapshot
I|<seq>|<ts>|<class>|<sec>|<n>|<B|A>:<px>:<qty>|…        increment, n levels
H|<ts>                                                   everything stamped <= ts is sent,
                                                         nothing more will be
E                                                        the channel has ended
```

### 4.2 Order channel: venue → engine

```
A|<cl_id>|<ts>|<ord_id>|<leaves>                          order accepted
R|<cl_id>|<ts>|<reason>                                  order rejected
F|<cl_id>|<ts>|<ord_id>|<trade_id>|<px>|<qty>|<leaves>   fill
X|<cl_id>|<ts>|<ord_id>                                 order cancelled
Y|<cl_id>|<ts>|<ord_id>|<reason>                        cancel refused
K|<ts>                                                  the account is described,
                                                        you may trade
H|<ts>                                                  horizon, as on the feed
E                                                       the channel has ended
```

## 4.3 Order channel: engine → venue

```
HELLO|<lookahead_ns>                                    handshake, arrives first
N|<cl_id>|<ts_seen>|<class>|<sec>|<B|S>|<px>|<qty>|<tif> new order
C|<cl_id>|<ts_seen>|<class>|<sec>|<ord_id>              cancel
TIME|<t>                                                I have processed everything up to t
BYE                                                     the engine is finished; no more orders
```

## 4.4 Fields

| field | type | what it is |
|---|---|---|
| `seq` | uint64 | per-instrument event number, shared by `S` and `I` |
| `ts` | uint64 | logical time, nanoseconds |
| `class` | string ≤ 7 chars | venue/class code |
| `sec` | string ≤ 31 chars | instrument code |
| `nbid` / `nask` / `n` | uint16 | how many levels follow |
| `px` | decimal | price |
| `qty` | uint64 | volume, an **integer** |
| `cl_id` | uint64 | the engine's transaction id, **opaque** |
| `ts_seen` | uint64 | the logical time at which the engine decided |
| `ord_id` | uint64 | order id, **issued by you** |
| `trade_id` | uint64 | trade id, **issued by you** |
| `leaves` | uint64 | how much of the order is still working |
| `reason` | uint32 | refusal code, see 5.6 |
| `tif` | uint8 | `0` = DAY, `1` = IOC, `2` = FOK |
| `lookahead_ns` | uint64 | the declared order latency, see section 6 |

# 5. Rules per message

## 5.1 `K` — nothing happens without it

The engine does not trade until you have said `K`. A venue that forgets it streams a perfect book into an engine that will never place a single order — and then the run ends on a silence timeout.

Send `K|<ts>` **once, first**, stamped with the time your dataset starts at. That stamp is the origin of the whole run (see 6.4).

## 5.2 The book: `S` and `I`

> **Publish `S`. Increments are NOT APPLIED by the engine today.**
>
> The `I` format is specified in full and is parsed down to the last field, but it is not yet applied to the book. Nothing crashes and nothing warns: the book simply stays as the last `S` left it, and the engine goes on trading against a frozen book.
>
> So a venue built on increments looks like it works and produces meaningless numbers. Until this changes, send every book change as a snapshot. The rules below describe `I` for the future, and for the case where you would rather write it correctly the first time.

- `S` and `I` share **one** `seq` space per instrument. An `S` numbered N is the state after event N; an `I` numbered N+1 applies on top of it.
- **The first message about an instrument is always `S`.** There is nothing for an increment to apply to.
- **A gap in `seq` is a bug**, not a loss in the network: TCP does not lose. Number them consecutively. Note that nobody currently cross-checks these numbers, so a gap will not announce itself — this is your own safety net rather than a check from outside.
- **Volume in an increment is absolute** — "there is now this much at this level", not "it changed by this much". That makes a repeat idempotent and keeps an error from accumulating over a million events.
- **A volume of exactly zero removes the level.** Exactly zero, not a comparison with a tolerance: this is a count, not a measurement.
- **One `I` is one atomic change to the whole book**, however many levels it names. The engine treats each of your market messages as a separate event, so "one message per level" would make it decide against intermediate books that never existed.
- **At most 64 levels a side** — the engine will not accept more. If a change reaches deeper than that, send an `S` rather than truncating silently.
- **Levels run best to worst**: bids by descending price, asks by ascending.
- In an `S`, all `nbid` bids come first, then all `nask` asks.

## 5.3 Orders: `A` / `R` / `F` / `X` / `Y`

**`cl_id` is opaque.** The engine keeps its own transaction bookkeeping in it, and its internal structure may change, so it is not something to build on. There is exactly one rule: **echo `cl_id` back byte for byte in every reply**. Do not parse it and do not use it as the key of an order — an order and its cancel carry different values.

**A cancel has two identifiers and that is not redundancy.** `cl_id` identifies the cancel itself; `ord_id` is what is being cancelled. Treat `cl_id` as the order's key and cancels break.

The rest:

- `A` comes strictly before any `F` / `X` for that order: `A` is where you issue the `ord_id`.
- `ord_id` is non-zero and does not change for the life of the order. **One exception, inbound only:** a `C` may carry `ord_id == 0`. That is not garbage — on the way out the engine pulls everything it has open, including an order you have not answered yet and for which it therefore has no id. Answer with `Y` and code `1`. Do not fall over.
- `R` arrives before an `ord_id` has been assigned, so there is none in it. That is normal.
- `cl_id == 0` is a protocol error; the engine never sends it.
- **`leaves` is mandatory and exact** in `A` and in every `F`. It is what tells the engine the order has finished. With an inexact remainder the engine will go on believing the order is alive and will not place the next one.
- After an `X`, or after an `F` with `leaves == 0`, **not one further message about that order**.
- `trade_id` is non-zero and **unique within the run**. The engine deduplicates fills on it; a repeat would be counted twice and every position derived from it would drift.
- **`Y` says nothing about the fate of the order.** It is still alive; what became of it arrives later through an `F` or an `X`. There is deliberately no "is it still alive" field: out of a refusal alone that is unknowable, and a field that is right half the time is worse than none.

## 5.4 `E` — the only clean end

Silence is not an end. The engine cannot tell a finished dataset from a venue that died: it will wait, and then report a successful run as a lost connection.

`E` is a horizon with the value "infinity": "nothing more on this channel, ever". Send it **on both channels**. The run ends when both have said `E` and both sides have worked through whatever was left.

Nothing may arrive on a channel after its `E`.

**The driver does this for you.** You may call `Session::Finish()` explicitly, but the session's destructor calls it anyway, including when you are leaving on an error: `E` means "nothing more on this channel", and that is true however you are leaving. A second call does nothing.

## 5.5 Prices

Write a price in ordinary decimal notation with a fixed number of digits, never in scientific notation.

This is not pedantry. A price on a tick grid is computed as `round(x / tick) * tick`, and the result regularly lands one least-significant bit away from the `double` nearest to the decimal you meant. Shortest round-trip printing writes that miss out in full: instead of `239.64` you get `239.64000000000001`, which no longer lies on any tick grid at all. `cmf_driver` prints nine digits and strips trailing zeros — the error sits far below the ninth digit and rounds away.

**Inside your venue, compare prices in whole ticks rather than `double`s for equality.** Two numbers that print the same can differ in a bit.

**`nan` and `inf` are not prices.** The standard parsers accept them, and a NaN does not trip a "bad price" check: it makes every comparison false at once, and whatever was resting on those comparisons quietly stops working.

### 5.6 Refusal codes

| code | meaning |
|---|---|
| 1 | no such order |
| 2 | the order is already terminal |
| 3 | the price is off the tick grid |
| 4 | invalid parameters |
| 5 | unknown instrument |
| 9 | internal error |

**Code `3` must be implemented.** The engine rounds every price to a tick grid of its own. If yours differs, every quote lands on a price your book does not have and nothing matches at all — silently, for the whole run. A venue must **refuse** an off-grid price rather than round it: that turns the quietest possible failure into a loud one on the very first order.

## 6. Time and synchronisation

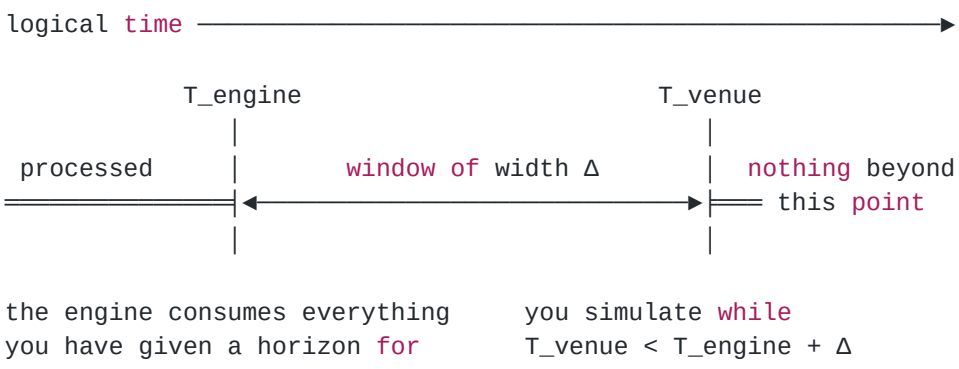
The most important section. The driver covers all of it, but you need to understand what it is doing.

### 6.1 Why

Without synchronisation: the engine gets a book as of instant 1000, answers it with an order, and while that order is in flight you have moved on to 1500. The decision was taken against one book and the execution computed against another. That fill is an invention, and the size of the invention depends on how busy the machine is. Run it twice and get two different results.

The problem is called **parallel discrete-event simulation**. The scheme used here is the conservative one of Chandy, Misra and Bryant.

### 6.2 The window




The bound is strict: `T_engine + Δ` is an instant you may run **up to** but never **into**. Why, in 6.3.

**Δ ( `lookahead_ns` )** is the declared order latency. It arrives in `HELLO`, one value for both sides. It is simultaneously a promise — "I will send you nothing that takes effect sooner than Δ after what I have seen" — and that promise is exactly what lets each side run ahead without asking permission for every event.

Latency stops being accidental and becomes **declared**, so it belongs next to the numbers a run produces: results are comparable only between runs at the same Δ.

## 6.3 Four obligations

**Yours:**

1. **Do not reach `T_engine + Δ`.** `T_engine` is the last value that arrived in a `TIME`.
2. **Stamp every message**, and announce `H|ts` on whichever channel currently has nothing to say. On **both** channels: there is no ordering between the two connections, and a promise about one says nothing about the other.

**`H|X` means "everything stamped ≤ X is sent, nothing stamped ≤ X will follow".** That is `≤`, not `<`. The engine uses the promise to take the **other** channel up to and including X; if a message at exactly X could still turn up on the quiet one afterwards, the processing order would be decided by the socket rather than by the model — and two runs of one dataset would diverge.

The same reasoning is **why the window's bound is strict**. The engine announces its time, may then process a message at exactly that time, and send an order — which takes effect at `announced + Δ`, that is, **exactly on the bound**. So on the bound itself an order may still be in flight, and two consequences follow, equally binding:

- **you may not step onto that instant yourself.** Otherwise one and the same order lands either before or after the event stamped at its own effective time — depending on when its bytes arrived — and matches against two different books. This has no symptom: the order fills in one run and is cancelled in another, both pictures are plausible, and everything after that diverges;
- **the order channel may only promise strictly below the bound.** Promising the bound itself means promising about an instant you know nothing about.

The feed likewise promises not the bound but where it stands: everything below its current stamp is published and the next thing is exactly at it. That is both true and the most you can say.

The price of strictness is that the window advances by `Δ − 1` per round trip rather than `Δ`. At any sane Δ that is nothing.

The driver does all of this for you; if you write your own transport, do not skip it.

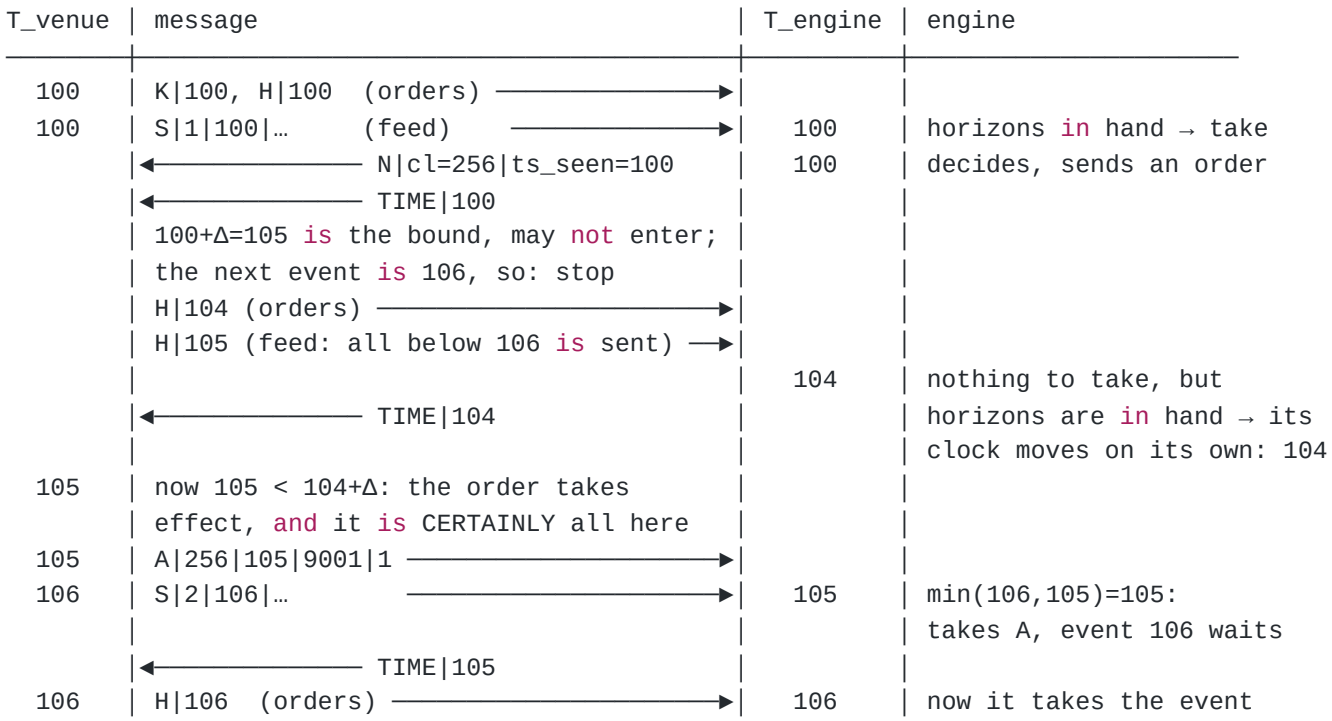
**The engine's** (it keeps them; you need to know them):

3. Never consume a message stamped later than the smaller of your two horizons.
4. Announce its own time: `ts_seen` on every order, and `TIME|t` when there are no orders.

## 6.4 The beginning

Before the first `TIME` the engine has no clock, and it cannot grant you permission to reach a time it has never heard of. So **the origin is yours to choose**: the stamp in `K` is the first instant the engine learns about, and up to it you may send everything you have at that stamp, having announced `H|<ts>` on both channels. From there the loop is closed: the engine reaches your stamp and announces its own `TIME`.

## 6.5 How the stamps travel




Two lines carry the whole point.

**The stop at 105.** The venue ran not into a lack of data but into the bound: an order stamped with the engine's last announced time takes effect at exactly 105, and until the engine has announced something above 100 there is no way to know it is not in flight. Having waited for `TIME|104`, the venue gets 105 whole — and hands the order over *before* it applies event 106, not after.

**Event 106 at the engine.** It arrived **earlier** than the engine learned that nothing below 106 would come on the order channel. Taking it straight away would risk a fill stamped 105.5 turning up afterwards. The `H` on the quiet channel closes that hole.

## 6.6 Why it does not deadlock

Suppose you are blocked: `T_venue ≥ T_engine + Δ`. Everything strictly below the bound has been sent and announced, so the engine is entitled to reach `bound − 1` — and it does, after which the bound moves `Δ − 1` further out.

Two things this breaks without, and neither is obvious:

- **Announce the limit you are allowed to reach, not the place where you stand.** The latter says nothing new after the first time, the engine's safe time never rises, and the two sides wait on each other. The promise is honest either way: everything strictly below the limit you have already sent, and any order still to arrive is stamped no earlier than the engine's last announcement and takes effect no earlier than the limit.
- **The engine moves its own clock even when there is nothing to process.** Having consumed everything up to its safe time, it *is* at that time. Otherwise a gap in the data would stop the run: it waits for events you are not allowed to produce, and you wait for an announcement it will never make.

This works **as long as Δ > 1**. A round trip moves the bound by `Δ − 1`, so Δ = 1 is a step of zero and a hang, and Δ = 0 collapses the window sooner still. The engine refuses to start with zero; one nanosecond makes no

sense as a declared latency either.

### 6.7 What synchronisation costs

A gap of width G in the data costs roughly `G / (Δ − 1)` exchanges of null messages: a round trip moves the window by `Δ − 1` (why not by Δ: see 6.3). On real tick data events are denser than any sensible Δ and none of this is noticeable. On a synthetic or very sparse feed it shows up immediately, and the cure is either a larger Δ or denser data.

## 7. Protocol errors

The engine stops the run rather than trying to continue, if:

| what | why it cannot continue |
|---|---|
| a stamp on a channel went backwards | a single queue does not reorder messages |
| a message stamped ≤ a horizon you already announced | the other channel has already been taken further on that promise; there is nothing to go back and replay |
| a stamp equal to `0` or `2^64-1` | both are reserved: `0` = "not stamped", the maximum = "the channel has ended" |
| a field that does not parse in full | `12x` yields `12`, and that is a different order |
| `cl_id`, `ord_id` or `trade_id` equal to zero **in your reply** | zero is a reserved value for each of them. In an INBOUND cancel `ord_id` may be zero; that is a different case, see 5.3 |
| more levels than `nbid` / `nask` / `n` announced | which of the two numbers is the truth is not the engine's to decide |
| `class` / `sec` longer than allowed | a truncated code is a different instrument |
| an unknown line tag | the two sides disagree about the protocol; there is nothing to resynchronise on |

This is not strictness for its own sake. A silently dropped fill is a position that is wrong for the rest of the run, and a result with nothing to trace it back to. An error that breaks the run immediately is cheaper than one that spoils the answer.

## 8. The driver

`cmf_driver.hpp` is header-only and depends on standalone asio and the standard library. On nothing else, deliberately: the file is meant to be dropped into someone else's project and compiled there.

It does: command-line parsing, binding and accepting both connections, framing, encoding and decoding of every message (including the price rule), and all of the synchronisation from section 6.

It does **not** do anything that is a venue's job: reading your dataset, the book, matching, or deciding whether a price lies on your grid.

## 8.1 The skeleton

```
#include "cmf_driver.hpp"

int main(int argc, char** argv) try {
    const auto options = cmf_driver::ParseOptions(argc, argv);
    cmf_driver::Session session(options);

    session.on_new_order = [&](const cmf_driver::NewOrder& order) {
        // order.effective_ts is when it takes effect. Never fill it earlier.
        session.Accept(order.client_id, order.effective_ts, my_book.Add(order), order.qty);
    };

    session.on_cancel = [&](const cmf_driver::Cancel& cancel) {
        // ord_id may be zero -- you must answer regardless, see 5.3. You have
        // no order numbered zero, the lookup misses, and the refusal branch
        // handles it by itself.
        if (my_book.Remove(cancel.order_id)) {
            session.Cancelled(cancel.client_id, cancel.effective_ts, cancel.order_id);
        } else {
            session.CancelRejected(cancel.client_id, cancel.effective_ts, cancel.order_id,
                                   cmf_driver::RejectReason::NoSuchOrder);
        }
    };

    const auto lookahead = session.Start();   // accept both, read HELLO
    session.Ready(first_event_ts);            // K plus the first horizons

    for (auto& event : my_dataset) {
        session.AdvanceTo(event.ts);          // waits for permission, drains orders
        if (session.EngineFinished()) break;

        session.PublishSnapshot(event.seq, event.ts, "CLASS", "SEC", bids, asks);
    }

    session.Finish();                         // E on both channels
    return 0;

} catch (const std::exception& error) {
    // The driver reports protocol violations by throwing. Catch it: without
    // this the first thing you see is `terminate called after throwing`,
    // without the text that says what is actually wrong.
    std::fprintf(stderr, "venue: %s\n", error.what());
    return 1;
}
```

## 8.2 What `AdvanceTo` does

The one call worth understanding end to end:

1. it parses everything the engine has already sent **into a queue** — not into your handlers;

2. while the requested time is not strictly inside the window, it announces horizons and **blocks**, waiting for a `TIME`;
3. and only then calls your handlers for everything that has come into effect by `ts`.

The order is exactly that, and step 3 follows step 2 for a reason: the queue is complete for the instant `ts` only once `ts` is strictly inside the window (see 6.3). Handing an order over earlier means handing it over either before or after the event stamped at that same instant, depending on when its bytes arrived.

So after `AdvanceTo(ts)` returns, everything the engine could have said about times up to and including `ts` has already been through your handlers.

### 8.3 Answering straight away or deferring

Both are allowed. You may answer from inside the handler (as the example does), or put the order in a queue of your own and answer when its time comes. The only requirement is that **no reply is stamped earlier than `effective_ts`**.

### 8.4 If you write your own transport

Taking the driver is not compulsory. In that case re-read section 6 in full and check yourself against three points: announce the limit rather than your current time; announce on both channels; announce **before** you block.

## 9. Integration checklist

- [ ] `--host`, `--feed-port`, `--order-port` are parsed; unknown arguments are ignored;
- [ ] both ports are bound **before** the dataset is loaded;
- [ ] `K|<ts>` is sent first;
- [ ] every message is stamped, and `0` and `2^64-1` are never used;
- [ ] `H` is announced on both channels before every block;
- [ ] `T_venue` is always **strictly below** `T_engine + Δ` — not "no greater than", see 6.3;
- [ ] `cl_id` is echoed back byte for byte;
- [ ] `ord_id` is issued in `A`, non-zero, unchanging;
- [ ] `trade_id` is non-zero and unique for the run;
- [ ] `leaves` is exact in `A` and in every `F`;
- [ ] after an `X` / an `F` with `leaves == 0`, silence about that order;
- [ ] a `C` carrying `ord_id == 0` does not bring the process down;
- [ ] an off-grid price is **refused** with code `3` rather than rounded;
- [ ] prices are compared in whole ticks;
- [ ] `E` is sent on both channels before leaving `main` — with the driver this is free, the `Session` destructor does it; your own transport has to remember.